

キャッシュを道具に組み込む ー 古いファイルを読む事故を防ぐ

2026-09-14 | Houdini 21.0.700 | /obj/smoke | ノード 5 / 接続 4

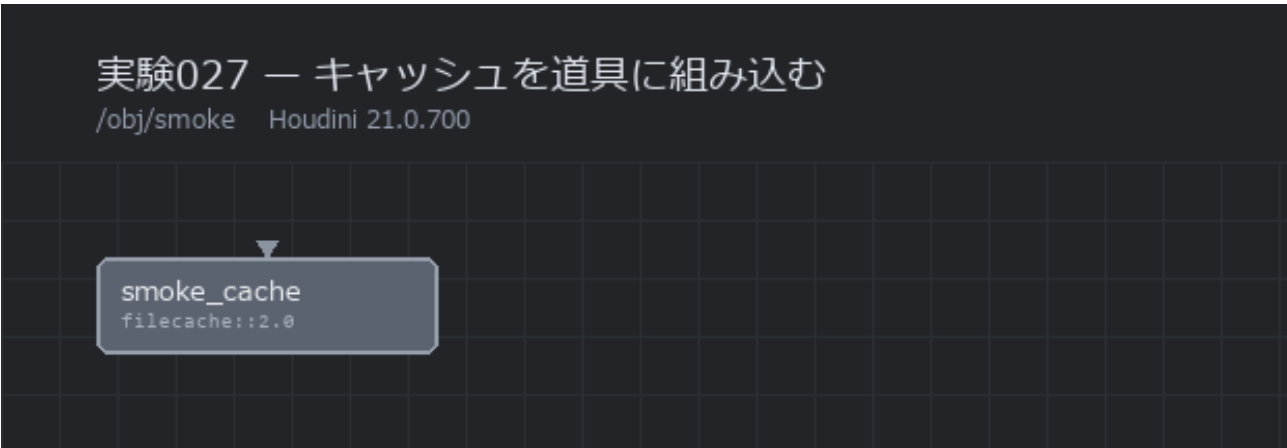
何を作ったか

実験025で filecache の挙動は測った。読み込みは速く、値は完全に一致する。ただし手で組むと毎回同じ手間がかかるうえ、もっと危ない落とし穴がある。

上流を変えたのに、古いキャッシュを読んでしまうことである。このとき画も数字も出てくる。エラーも出ない。ただ、それは前の設定の結果である。測った値が静かに嘘になる。このプロジェクトで一番避けたい事故がこれになる。

そこで hou_tools.cache_sim()
に指紋を持たせた。上流のノードの種類と「既定から動かしたパラメータ」をまとめて SHA-1 を取り、キャッシュの隣に置く。一致したときだけ読み、変わっていれば黙って計算し直す。仕組みが効いているかを、3つの形で確かめた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.06

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.05 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 100 件
smoke_cache	filecache::2.0	f1 = 1.0 f2 = 30.0 file = D:/Claude/houdini/out/027_cache/smoke.0001.bgeo.sc filemethod = 1 folder2 = 1 folder3 = 1 loadfromdisk = 1

1. キャッシュを通して値は変わらないか

「ほぼ同じ」ではなく「完全に同じ」を求める。ここが崩れると、以降の測定がすべて信用できなくなる。

フレーム	計算した値	読んだ値	差
1	4,230.662863	4,230.662863	0.0000000000
10	30,202.378274	30,202.378274	0.0000000000
20	37,789.693870	37,789.693870	0.0000000000
30	43,739.861589	43,739.861589	0.0000000000

2. 速さと、2回目の振る舞い

2回目は同じ設定で呼び直した。指紋が一致するので、書き出しは起きないはずである。

やったこと	書き出し	読み出し	結果
計算しながら30フレーム回す	—	0.83 秒	これが基準
1回目の cache_sim()	0.12 秒	0.15 秒	計算より 5.5倍速い
2回目の cache_sim() (同じ設定)	起きない	0.20 秒	1回目と完全に一致

3. 上流を1つ変えたら、気づくか

pyrosolver の dissipation を、既定の 0.1 から 0.05 に下げた。それ以外は何も触っていない。



Houdini

dissipation 0.1 (既定) 。 F30。



Houdini

dissipation
0.05。同じカメラ・同じフレーム。頭が濃く大きい。

調べたこと	結果
指紋	8f35eab0ca26… → 366303d3368d… (変わった)
cache_sim() の判断	fingerprint changed → 計算し直した
値が変わったフレーム	29 / 30
F30 の密度の合計	43,739.86 → 66,890.32 (+52.9%)
指紋のファイルを壊した場合	no fingerprint → 計算し直した

わかったこと

- ・ キャッシュを通しても値は1ビットも変わらなかった。30フレームすべてで差が 0.0000000000。025の結論が、道具に組み込んだあとでも保たれている。
- ・ 2回目は書き出しが起きなかった。指紋が一致したので読むだけになり、値も1回目と完全に一致した。使う側は毎回 cache_sim() を呼ぶだけでよい。
- ・ 上流を1つ動かしたら、指紋が変わって計算し直した。これが無ければ、30フレーム中29フレームで違う値を、正しい値として読んでいたことになる。F30 で +52.9% のずれ。
- ・ 指紋は上流のノードをたどって、種類と「既定から動かしたパラメータ」だけを集めて作る。既定値を含めないで、Houdiniの版が上がって既定が増えても指紋は暴れない。
- ・ 指紋のファイルが壊れていたら、読まずに計算し直す。壊れたJSONを置いて確かめた。「読めないときは安全側に倒す」ー迷ったら計算し直すほうが、静かに間違えるよりよい。
- ・ 同じ計算を2回書き出すと、ファイルの合計サイズが 138バイト違った (38,927,958 → 38,928,096) 。中身の値は同じでも、ファイルはバイト単位では一致しない。つまりキャッシュの正しさをファイルの比較で確かめることはできない。指紋を「ファイル」ではなく「ネットワーク」から作っている理由がこれ。
- ・ 書き出したファイルの数がフレーム数と違えば、cache_sim() は例外を投げて止まる。実験025で40フレームのつもりが 240フレーム書かれて 3.5GB を無駄にした件を、道具の側で塞いだ。
- ・ pyrosolver の dissipation は既定で有効・値は 0.1。何も触らなくても煙は薄くなり続けている。半分にしかたけで F30 の密度が +52.9% になる。「既定=何もしていない」ではない。

- ・ hython は、レンダしたあとにシーンを作り直すと固まる。今回この順で10分待っても進まなかった。レンダは1プロセスの最後に1枚だけにして、比較の2枚目は別プロセスで撮る。カメラをそろえるため、枠を JSON に残して次のプロセスが読む。

次にやること

- ・ 重い実験 (Karma、FLIP、大きな Pyro) を `cache_sim()` に載せ替える
- ・ 028 として、まだ触っていない分野に進む
- ・ 実験030に達したら、001~030の振り返り点検を行う